

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ
КЫРГЫЗСКОЙ РЕСПУБЛИКИ
КЫРГЫЗСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ им. И.Раззакова**

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

Кафедра: Программное обеспечение компьютерных систем

Курсовой проект

по дисциплине: «Тестирование ПО»

**по теме: «Автоматизированная система учета примерок розничного
магазина по продаже одежды»**

Выполнил: студент группы ПИ-2-21

Копжашаров Азамат Таалайбекович

Проверила: Мусина Индира Рафиковна

Бишкек 2024

Оглавление

Введение.....	4
Часть 1. АНАЛИЗ И РАЗРАБОТКА ТРЕБОВАНИЙ.....	5
1.1. Актуальность работы.....	5
1.2. Цель работы.....	5
1.3 Спецификация проблемы.....	5
1.4 Назначение и цели создания системы.....	6
1.4.1 Назначение.....	6
1.4.2 Цели.....	6
1.5 Требования к системе.....	6
1.5.1 Требования к функциям:.....	6
1.5.2 Требования по безопасности.....	6
1.5.3 Требования к надежности.....	7
1.5.4 Требования к информационной системе.....	7
1.5.5 Требования к клиентской части.....	7
1.5.6 Требования к серверной части.....	7
1.5.7 Требования к организационному обеспечению.....	7
Часть 2. ПЛАНИРОВАНИЕ ТЕСТИРОВАНИЯ.....	9
2.1. Введение.....	9
2.2. Описание объектов тестирования (область тестирования).....	9
2.3. Определение целей и задач тестирования.....	9
2.3.1. Цели тестирования:.....	9
2.3.2. Задачи тестирования:.....	9
2.4. Стратегии тестирования.....	10
2.5. Начальные условия (пререквизиты).....	10
2.6. Приоритеты тестирования.....	10
2.7. Методы тестирования.....	10
2.8. Роли и обязанности.....	11
2.9. Процедуры управления тестами:.....	11
2.10. Результаты.....	11
2.11. График тестирования.....	11

2.12. Тестовые сценарии.....	12
2.13. Среда тестирования.....	13
2.14. Риски тестирования.....	14
Часть 3. Результаты тестирования.....	15
3.1. Введение.....	15
3.2. Unit – тестирование Введение.....	15
3.3. Интеграционное тестирования.....	24
3.4. Тестирование пользовательского интерфейса.....	25
3.5. Метрики кода.....	26
3.7. Статический анализ кода.....	27
3.8. Профилирование программы.....	30
Часть 4. Заключение.....	33

Введение

I-Tech International – компания-разработчик, которая специализируется на создании программного обеспечения для автоматизации сложных бизнес-процессов. Основное направление деятельности компании – разработка высокотехнологичных решений для автоматизации швейного производства и логистики, включая складской учет. Инновационные разработки I-Tech International помогают предприятиям оптимизировать работу, повысить эффективность производства и улучшить управление цепочками поставок.

Компания активно занимается автоматизацией швейного производства, предлагая решения для планирования, контроля и управления ресурсами на швейных фабриках. Это способствует снижению производственных затрат, повышению качества продукции и увеличению производительности. Еще одним важным направлением является автоматизация логистики и складского учета. Системы, разработанные компанией, позволяют автоматизировать ключевые складские операции, что повышает точность учета, снижает затраты и улучшает обслуживание клиентов.

Функционал системы предназначен для менеджеров-продавцов для автоматизации работы с документацией примерок. Прделанная работа заключалась в разработке функций создания и сохранения, детального редактирования, удаления, просмотра и общий список документов примерок, а также создания дочерних документов других видов на основе исходного документа примерки.

Цель разработки заключалась в том, чтобы автоматизировать работу с документацией примерок для менеджеров-продавцов. Это позволит намного меньше тратить времени на ручное заполнение документов, позволит хранить документы примерки в электронной базе данных, что ускорит поиск специфичных документов, сбор информации по примеркам и дальнейший анализ примерок.

Часть 1. АНАЛИЗ И РАЗРАБОТКА ТРЕБОВАНИЙ

1.1. Актуальность работы

В условиях высокой конкуренции на рынке розничной торговли одеждой и особыми требованиями к персонализированному подходу к клиентам, важно обеспечивать не только качественное обслуживание, но и эффективно управлять бизнес-процессами, связанными с продажей и арендой товаров. Особую актуальность приобретает управление процессами примерки, поскольку примерка является ключевым этапом, напрямую влияющим на решение клиента о покупке или аренде товара. Однако в существующих системах управления розничным магазином часто отсутствует функция автоматизации этого процесса, что приводит к трудностям в оформлении документации, увеличению временных затрат на обслуживание клиентов и повышенной вероятности ошибок.

Внедрение документа примерки в систему «Krista Store» позволяет существенно сократить время обработки информации, повысить точность данных и обеспечить лучшее взаимодействие между клиентом и магазином. Автоматизация процесса примерки не только ускоряет создание документов, но и упрощает управление бронированием и продажами после примерок, улучшая общую эффективность бизнес-процессов. Таким образом, добавление данной функции в систему актуально для оптимизации работы магазина, увеличения уровня удовлетворенности клиентов и повышения продаж, что особенно важно для компаний, стремящихся к удержанию своей конкурентоспособности на рынке.

1.2. Цель работы

Целью работы является разработка и внедрение функционала управления документами примерок в систему Krista Store. Основная задача заключается в создании инструментов, которые позволят продавцам:

- Просматривать информацию о прошедших примерках;
- Создавать, редактировать и удалять документы примерок;
- Создавать, редактировать и удалять документа брони и продажи на основе документа примерок.

Данный функционал позволит продавцам более эффективно вести документацию примерок в магазине, ускорить обслуживание клиентов и хранить документы в электронном виде, для быстрого доступа и поиска.

1.3 Спецификация проблемы

В ходе анализа существующей системы были выявлены следующие проблемы:

- Хранение документов примерки в бумажном виде, что усложняет сбор данных о примерках и их последующий анализ;
- Отсутствие связи между бумажными документами примерки и уже существующими в электронной базе данных документами брони, продажи;
- Медленное ручное оформление документов примерок, что может привести к потенциальным потерям в продаже.

1.4 Назначение и цели создания системы

1.4.1 Назначение

Доработка системы KristaStore предназначена для улучшения процессов управления документацией примерок. Это позволит сотрудникам компании более эффективно записывать и отслеживать примерки платьев, а также автоматически генерировать документы для последующей продажи или бронирования платьев на основе данных примерок.

1.4.2 Цели

Целью доработки системы KristaStore является разработка функционала для управления документацией примерок, а также оптимизация внутренних бизнес-процессов, связанных с документами продажами и бронированиями.

Основными целями создания функционала были:

- Упрощение создания и редактирование документа примерки;
- Обеспечение простого и быстрого доступа к истории примерок.

1.5 Требования к системе

1.5.1 Требования к функциям:

- Создания нового учета примерки;
- Редактирование учета примерки;
- Удаление учета примерки;
- Создание дочернего учета на основе учета примерки:
 - Создание документа продажи на основе документа примерки;
 - Создание документа бронирования на основе документа примерки;
- Просмотр документа примерки по его серийному номеру;
- Просмотр списка документов примерок;
- Фильтрация списка документов примерок;
- Сортировка списка документов примерок;

1.5.2 Требования по безопасности

Система должна обеспечивать надежную защиту данных клиентов, включая:

- Авторизацию и аутентификацию пользователей;
- Контроль доступа к клиентской информации;
- Защиту данных от несанкционированного доступа и утечки.

1.5.3 Требования к надежности

Система должна сохранять работоспособность и обеспечивать восстановление своих функций при возникновении следующих внештатных ситуаций:

- при сбоях в системе электроснабжения аппаратной части, приводящих к перезагрузке ОС, восстановление программы должно происходить после перезапуска ОС и запуска исполняемого файла системы;
- при ошибках в работе аппаратных средств (кроме носителей данных и программ) восстановление функции системы возлагается на ОС;
- при ошибках, связанных с программным обеспечением (ОС и драйверы устройств), восстановление работоспособности возлагается на ОС.

1.5.4 Требования к информационной системе

- Используемое при разработке программное обеспечение, библиотеки программных кодов, СУБД должны иметь широкое распространение.
- В качестве основы для разработки должна использоваться платформа Visual Studio.
- В качестве серверной операционной системы должна использоваться ОС семейства Windows NT.
- В качестве основы должен использоваться фреймворк ASP.NET, язык программирования C#.

1.5.5 Требования к клиентской части

Клиент должен использовать один из следующих веб браузеров:

- Google Chrome, Mozilla Firefox, Yandex Browser, Opera – 25 версии и выше;
- Internet Explorer 10 версии или выше;

1.5.6 Требования к серверной части

Сервер должен удовлетворять следующим критериям:

- Операционная система семейства Windows NT;
- Кроссплатформенный веб-сервер
- СУБД Postgresql

1.5.7 Требования к организационному обеспечению

Организационное обеспечение системы должно быть достаточным для эффективного выполнения персоналом возложенных на него обязанностей при

осуществлении автоматизированных и связанных с ними неавтоматизированных функций системы.

Заказчиком должны быть определены должностные лица, ответственные за:

- Обработку и контроль вводимой информации в систему;
- Администрирование информационной системой.

Часть 2. ПЛАНИРОВАНИЕ ТЕСТИРОВАНИЯ

2.1. Введение

Этот документ описывает план тестирования для модуля управления документацией примерок для системы автоматизации розничного магазина по продаже одежды. Полная стратегия тестирования системы состоит из следующих типов испытаний и выполняется в следующем порядке:

1. Тестирование компонентов (unit testing). Тестируются все компоненты программного обеспечения.
2. Интеграционное тестирование. Тестирование программного обеспечения с целью убедиться в том, что компоненты правильно взаимодействуют между собой.
3. Тестирование пользовательского интерфейса. Проверка удобства и корректности отображения интерфейса.
4. Системное тестирование. Тестирование программного обеспечения в эмулированной производственной среде с целью проверки его функциональности.

2.2. Описание объектов тестирования (область тестирования)

Модуль управления учетами примерок:

- Создание учета примерки;
- Редактирование учета примерки;
- Удаление учета примерки.

База данных:

- Сохранение и управление данными о примерках клиентов.

Пользовательский интерфейс:

- Реализация фильтрации и сортировки данных.

2.3. Определение целей и задач тестирования

2.3.1. Цели тестирования:

- Проверить корректность работы ключевых функций системы;
- Обеспечить соответствие работы системы функциональным требованиям;
- Обнаружить и устранить возможные ошибки в реализации;
- Проверить удобство и интуитивность пользовательского интерфейса;

2.3.2. Задачи тестирования:

1. Проверить корректность работы основных операций управления учетами примерок (создание учета, редактирование учета, удаление учета);
2. Проверить корректную работу пользовательского интерфейса путем тестирования фильтрации и сортировки списка учетов примерок;

3. Проверить состояние базы данных во время тестов и после их завершения.

2.4. Стратегии тестирования

Для данной системы будут применены следующие виды тестирования:

- Функциональное тестирование;
- Тестирование безопасности;
- Тестирование производительности;
- Юзабилити-тестирование.

2.5. Начальные условия (пререквизиты)

Тестовая среда:

- Серверная часть приложения на C# (backend);
- Клиентская часть приложения (frontend), доступный через браузер, с подключением к серверной част;
- База данных PostgreSQL с тестовой схемой данных;
- Настроенные тестовые данные в базе (клиенты, товары, продавцы и т.п.).

2.6. Приоритеты тестирования

Следующие проверки перечислены в порядке уменьшения приоритетного уровня (первое имеет самый высокий приоритет):

- Функции – все ли заданные функции программы выполняются как ожидалось при их описании в Техническом Задании?
- Удобство и простота использования – действительно ли программное обеспечение является дружелюбным по отношению к пользователю?
- Защита – данные защищены?
- Производительность – программное обеспечение соответствует согласованному критерию выполнения?

2.7. Методы тестирования

Будут использоваться следующие техники тестирования:

- Тестовые сценарии – сценарии вариантов использования (с предопределённым вводом и ожидаемыми выходными данными);
- Проверка на удобство использования ПО - действия, чтобы оценить простоту системы в использовании;
- Ручное тестирование;

2.8. Роли и обязанности

Определяются следующие роли и обязанности:

- Лидер проверки качества (QA-quality assurance lead) - человек, ответственный за процесс планирования тестирования и его выполнение;
- Тестировщик - выполняет действия по тестированию, определенные в плане тестирования;
- Менеджер ПО (Product manager) - гарантирует, что тесты выполняются успешно с точки зрения пользователя;
- Техническая поддержка тестирования - гарантирует, что техническое оборудование на месте и помогает в течение испытаний.

2.9. Процедуры управления тестами:

- **Создание тестов:** Все тестовые сценарии и данные создаются на основе функциональных требований системы.
- **Управление дефектами:**
 - Обнаруженные дефекты фиксируются в системе трекинга (например, Jira или аналог).
 - Каждому дефекту присваивается уровень приоритета и назначается разработчик для исправления.
 - Исправленные дефекты проверяются повторным тестированием.
- **Регрессионное тестирование:**
 - Проводится после исправления дефектов для проверки, что изменения не повлияли на другие функции системы.
 - Используются заранее подготовленные тестовые сценарии для основных функций.

2.10. Результаты

После тестирования должны быть в наличии следующие документы:

- План тестирования – этот документ со всеми изменениями, сделанными в ходе процесса тестирования;
- Запросы на изменение – документ, описывающий изменения ПО, вызванные изменением требований или обнаруженными дефектами в течение испытания;
- Заключительный отчёт, подписанный заказчиком, подтверждающий, что система отвечает всем функциональным требованиям и требованиям качества.

2.11. График тестирования

Задание тестирования	Начало	Конец
Модульное тестирование	09.12.2024	16.12.2024
Интеграционное тестирование	16.12.2024	20.12.2024
Нагрузочное тестирование	20.03.2024	21.12.2024
Приёмочные испытания (Общее системное	21.12.2024	22.12.2024

тестирование)		
---------------	--	--

2.12. Тестовые сценарии

Следующие сценарии тестирования предназначены для проверки каждой программной функции. Часть этих сценариев повторно описывают сценарии вариантов использования.

Каждый сценарий тестов состоит из следующего:

- Описание - эта часть представляет повторное описание документа по используемому сценарию;
- Начальные данные – обычно это исходная конфигурация базы данных;
- Шаги тестирования — это действия, которые тестирующие должны выполнить в течение тестирования.
- Тестовые варианты – исходные данные и ожидаемая реакция ПО для каждого теста.

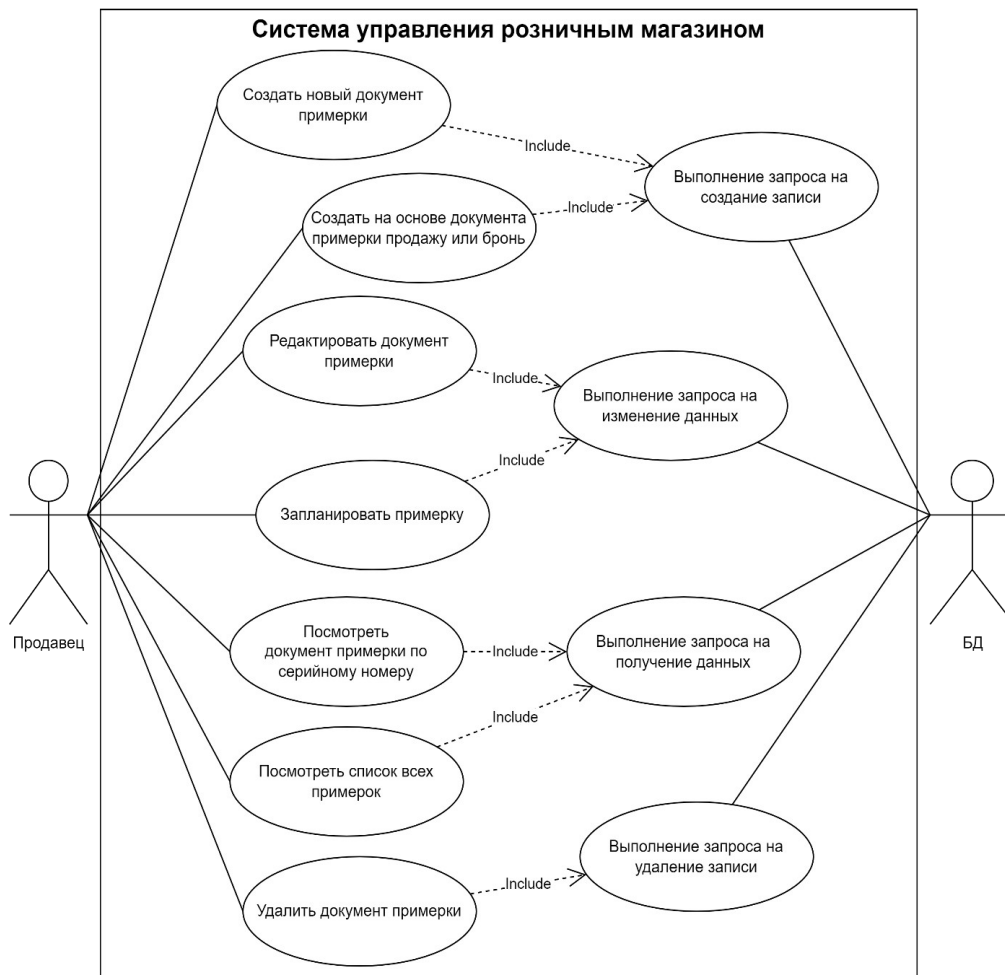


Рисунок 1 Диаграмма вариантов использования

2.13. Среда тестирования

Следующее программное обеспечение и аппаратная конфигурация должны быть доступны во время тестирования ПО. Одна рабочая станция со следующей конфигурацией:

- CPU: ЦП AMD Ryzen 3 2300U with Radeon Vega Mobile Gfx; Базовая скорость: 4,00 ГГц; Сокетов: 1; Ядер: 4; Логических процессоров: 4;
- RAM: 16 GB DDR4 (2400 МГц);
- SSD: 100 GB;
- GPU: AMD Radeon (TM) Vega 6 Graphics: Версия драйвера: 30.0.13017.5006: Дата релиза драйвера: 17.09.2021; Версия DirectX: 12 (FL 12.1);
- Установленная ОС: Windows 10, 11 (x64 разрядная ОС);
- Установленный .NET 8.0 и выше;
- Локальная база данных для тестирования приложения на базе PostgreSQL версии не ниже 16.1;
- Браузер Google Chrome, Opera, Firefox, Microsoft Edge, Yandex, Brave или др

2.14. Риски тестирования

Риск	Описание	Уменьшение
Нехватка времени (увеличение сроков тестирования)	Тестирование может занять больше времени, чем запланировано, из-за сложности задач, ошибок в тестируемом ПО или нехватки ресурсов.	Заложить дополнительное время на тестирование в плане проекта, а также предусмотреть возможность привлечения дополнительных тестировщиков в случае необходимости.
Недоступность ресурсов или данных	Отсутствие доступа к тестовым данным, необходимому оборудованию или инструментам может задержать процесс тестирования.	Заранее определить и подготовить необходимые ресурсы и доступы, а также предусмотреть резервные источники данных или оборудования, чтобы минимизировать влияние подобных задержек.
Тест – план не привязан к плану проекта	Тестирование и разработка сидят на одном общем проектном ресурсе — на времени. Если планы двух направлений не связаны жёстко (лучше всего на уровне одного общего плана работ по проекту, буквально линками между задачами в MS Project или любой другой подобной системе) или не синхронизированы на постоянной основе, существует вероятность или риск, что сдвиг планов	Необходимо формировать единый план проекта таким образом, чтобы разработка ПО и тестирование ПО были на одном уровне важности, и были очень жёстко связаны. Проектный менеджер не должен разделять эти два понятия, сочиняя план “независимо”. В таком случае будет чётко видно, что какая-то

	разработки (который влияет на дату поставки версии в тестирование) не будет учтён в плане работ по тестированию, что приведёт к недостатку времени на тестирование и, как следствие, к не завершению этапа тестирования в срок, что повлечёт за собой урезание самих тестов, либо растягивание проекта на больший срок выполнения.	часть работ по тестированию будет вылазить за deadline в общем плане проекта или на диаграммах (к примеру на диаграмме Ганта или сетевом графике).
--	---	---

Часть 3. Результаты тестирования

3.1. Введение

Данный отчет составлен согласно плану тестирования модуля управления учетами примерок для системы автоматизации розничного магазина

Информация о проекте:

Тестируемая система предназначена для автоматизации процессов управления документацией примерок, включающей такие функции, как просмотр списка документов примерок, создания учета примерки, редактирование учета и удаление учета примерки.

Основная цель системы – упрощение и ускорение работы с документацией примерок, за счет автоматизации процесса, а также повышение эффективности работы менеджеров и продавцов.

Заказчик документа: Мусина Индира Рафиковна

Автор документа: Копжашаров Азамат Таалайбекович

Сроки проведения тестирования: с 9 декабря 2024 года до 22 декабря 2024 года.

Длительность проведения: 14 дней

Участники тестирования: Копжашаров Азамат Таалайбекович

3.2. Unit – тестирование

Введение

Следующие сценарии тестирования предназначены для проверки каждой программной функции. Часть этих сценариев повторно описывают сценарии вариантов использования.

Каждый сценарий тестов состоит из следующего:

Описание - эта часть представляет повторное описание документа по используемому сценарию;

Начальные данные;

Шаги тестирования — это действия, которые тестировщики должны выполнить в течение тестирования.

Тестовые варианты – исходные данные и ожидаемая реакция ПО для каждого теста.

Тестовый сценарий №1. Создание документа примерки

Описание: Продавец хочет создать новый учет примерки в системе

Исходные данные: Тестовая база данных, данные о клиенте, товарах

Шаги тестирования:

После успешной авторизации, продавец выбирает пункт «Примерки» в главном меню; Продавец нажимает на кнопку «Создать документ примерки».

После открытия страницы с формой для создания учета примерки, продавец заполняет информацию: клиент, скидочная карта, скидка, ответственный продавец, товары (код товара), дата примерки;

Тестовые варианты:

Входные данные	Ожидаемый результат	Что проверяется
Клиент: Клиент 1 Сикдочная карта: 0000000000000001 Ответственный продавец: Продавец 1 Товары (штрих-код): 123 Дата примерки: 01.01.2025	Документ примерки будет успешно создан и сохранен в базе данных со статусом «Новый».	Успешное добавление нового учета в базу данных, при корректном вводе всех необходимых данных.
Клиент: Клиент 1 Скидка: 5% Ответственный продавец: Продавец 1 Товары (штрих-код): 123 Дата примерки: 01.01.2023	Документ примерки не будет создан. Вывод сообщения о том, что дата примерки не может быть в прошлом.	Ошибка при некорректном вводе входных данных.
Клиент: Клиент 1 Скидка: -5% Ответственный продавец: Продавец 1 Товары (штрих-код): 123 Дата примерки: 01.01.2025	Документ примерки не будет создан. Вывод сообщения о том, что скидка не может быть отрицательной.	Ошибка при некорректном вводе входных данных.
Клиент: -- Скидка: -- Ответственный продавец: -- Товары (штрих-код): -- Дата примерки: --	Документ примерки не будет создан. Вывод сообщения о том, что входные данные не полностью заполнены.	Ошибка при неполном вводе входных данных.
Клиент: -- Скидка: 5% Ответственный продавец: Продавец 1 Товары (штрих-код): 123 Дата примерки: 01.01.2023	Документ примерки не будет создан. Вывод сообщения о том, что поле клиента не может быть пустым.	Ошибка при некорректном вводе входных данных.
Клиент: Клиент 1 Скидка: 5% Ответственный продавец: -- Товары (штрих-код): 123 Дата примерки: 01.01.2023	Документ примерки не будет создан. Вывод сообщения о том, что поле продавца не может быть пустым.	Ошибка при некорректном вводе входных данных.

```
[Fact]
public async Task Should_CreateFittingDocument() {
    // Arrange
    var requestTime = DateTimeOffset.UtcNow.AddDays(3);
    var command = new CreateFittingDocumentCommand(1, 1);
```



```

// Act
var documentId = await _sut.HandleAsync(command, new());

// Assert
documentId.Should().NotBeEmpty();
var createdDocumentAssertion = Should.Assert<FittingDocument>(
    createdDocument => {
        createdDocument.CreatedByEmployeeId.Should().Be(1);
        createdDocument.WarehouseId.Should().Be(1);
        createdDocument.PlannedFittingDate.Should().BeCloseTo(requestTime, TimeSpan.FromHours(1));
        createdDocument.DocumentTypeId.Should().Be("fe879c1a-869a-48e2-8e91-af3b19b75d08");
        createdDocument.StatusId.Should().Be(DocumentStatuses.New);
        createdDocument.CurrencyId.Should().Be("3b5e3e8a-7921-476d-8b69-21c04ebcf7cc");
    }
);

_db.Verify(x => x.AddAsync(Should.Assert(createdDocumentAssertion), default), Times.Once);
_db.Verify(x => x.SaveChangesAsync(It.IsAny<CancellationToken>()), Times.Once);
}

public static TheoryData<CreateFittingDocumentCommand, string?, string> ValidationData => new() {
    {
        new(0, 1),
        nameof(CreateFittingDocumentCommand.EmployeeId),
        "Employee not found"
    }, {
        new(2, 1),
        nameof(CreateFittingDocumentCommand.EmployeeId),
        "Employee not found"
    }, {
        new(1, 0),
        nameof(CreateFittingDocumentCommand.WarehouseId),
        "Warehouse not found"
    }, {
        new(1, 4),
        nameof(CreateFittingDocumentCommand.WarehouseId),
        "Warehouse not found"
    }
};

[Theory]
[MemberData(nameof(ValidationData))]
public async Task Command_Should_BeValidatedCorrectly(
    CreateFittingDocumentCommand command,
    string? property,
    string reason
) {
    // Arrange
    _db.Setup(x => x.Employees).ReturnsDbSet([new(1, "", 1, true)]);

    // Act
    var sut = new CreateFittingDocumentCommandValidator(_db.Object);
    var validationResult = await sut.TestValidateAsync(command);
    _testOutputHelper.WriteLine(reason);

    // Assert
    if (string.IsNullOrEmpty(property)) {
        validationResult.ShouldNotHaveAnyValidationErrors();
    } else {
        validationResult.ShouldHaveValidationErrorFor(property);
    }
}

```

Тестовый сценарий №2. Редактирование документа примерки

Описание: Продавец хочет отредактировать существующий учет примерки в системе

Исходные данные: Тестовая база данных, данные о клиенте, товарах

Шаги тестирования:

После успешной авторизации, продавец выбирает пункт «Примерки» в главном меню;
Продавец выбирает нужный документ, который необходимо отредактировать и нажимает на кнопку редактирования.

После открытия страницы с формой для редактирования учета примерки, продавец изменяет нужную информацию: клиент, скидочная карта, скидка, ответственный продавец, товары (код товара), дата примерки;

Тестовые варианты:

Входные данные	Ожидаемый результат	Что проверяется
Клиент: Клиент 2 Скидочная карта: 0000000000000001 Ответственный продавец: Продавец 1 Товары (штрих-код): 123 Дата примерки: 01.01.2025	Документ примерки будет успешно отредактирован и сохранен в базе данных.	Успешное изменение существующего учета примерки, при корректном вводе всех необходимых данных.
Клиент: -- Скидочная карта: 0000000000000001 Ответственный продавец: Продавец 1 Товары (штрих-код): 123 Дата примерки: 01.01.2025	Документ примерки не будет отредактирован. Вывод сообщения о том, что необходимо заполнить все поля	Ошибка при неполном вводе всех входных данных.
Клиент: Клиент 2 Скидочная карта: 0000000000000001 Ответственный продавец: Продавец 1 Товары (штрих-код): 123 Дата примерки: 01.01.2023	Документ примерки не будет отредактирован. Вывод сообщения о том, что дата примерки не может быть в прошлом.	Ошибка при некорректном вводе входных данных.
Клиент: Клиент 1 Скидочная карта: 0000000000000001 Ответственный продавец: Продавец 99 Товары (штрих-код): 123 Дата примерки: 01.01.2025	Документ примерки не будет отредактирован. Вывод сообщения о том, что продавец с таким идентификатором не существует.	Ошибка при некорректном вводе входных данных.
Клиент: Клиент 99 Скидочная карта: 0000000000000001 Ответственный продавец: Продавец 1 Товары (штрих-код): 123 Дата примерки: 01.01.2023	Документ примерки не будет отредактирован. Вывод сообщения о том, что клиент с таким идентификатором не существует. быть в прошлом.	Ошибка при некорректном вводе входных данных.

```

[Fact]
public async Task Should_UpdateFittingDocument() {
    // Arrange
    var command = new UpdateFittingDocumentCommand {
        FittingDocumentId = new("e3dbe1d7-0ef7-4a15-837f-11042c2d4741"),
        EmployeeId = PatchOp.Update(1),
        ClientId = PatchOp.Update(new Guid("9b6619e0-3ef2-423e-bfc6-648c05cf4331")),
        FittingDate = PatchOp.Update(DateTimeOffset.UtcNow.AddDays(3))
    };

    // Act
    await _sut.HandleAsync(command, new());

    // Assert
    var fittingDocument = _db.Object.FittingDocuments
        .First(x => x.DocumentId == new Guid("e3dbe1d7-0ef7-4a15-837f-11042c2d4741"));

    fittingDocument.CreatedByEmployeeId.Should().Be(1);
    fittingDocument.ClientId.Should().Be(new("9b6619e0-3ef2-423e-bfc6-648c05cf4331"));
    fittingDocument.PlannedFittingDate.Should().BeCloseTo(
        DateTimeOffset.UtcNow.AddDays(3),
        TimeSpan.FromMinutes(3)
    );

    _db.Verify(x => x.SaveChangesAsync(It.IsAny<CancellationToken>()), Times.Once);
}

public static TheoryData<UpdateFittingDocumentCommand, string?, string> ValidationData => new() {
    {
        new() { FittingDocumentId = Guid.Empty },
        nameof(UpdateFittingDocumentCommand.FittingDocumentId),
        "Fitting document id is required"
    }, {
        new() { FittingDocumentId = Guid.NewGuid(), EmployeeId = PatchOp.Update(0) },
        "EmployeeId.Value",
        "Employee id should be greater than 0"
    }, {
        new() { FittingDocumentId = Guid.NewGuid(), EmployeeId = PatchOp.Update(2) },
        "EmployeeId.Value",
        "Employee does not exist"
    }, {
        new() { FittingDocumentId = Guid.NewGuid(), DiscountCardCode = PatchOp.Update("") },
        "DiscountCardCode.Value",
        "Discount card code should not be empty"
    }, {
        new() { FittingDocumentId = Guid.NewGuid(), DiscountPercent = PatchOp.Update(-1m) },
        "DiscountPercent.Value",
        "DiscountPercent should not be empty"
    }, {
        new() {
            FittingDocumentId = Guid.NewGuid(),
            DiscountCardCode = PatchOp.Update("0000000000000001"),
            DiscountPercent = PatchOp.Update(10m)
        },
        "DiscountCardCode",
        "Can't set discount card code and discount percent simultaneously"
    }, {
        new() { FittingDocumentId = Guid.NewGuid(), ClientId = PatchOp.Update(Guid.Empty) },
        "ClientId.Value",
        "Client id should not be empty"
    }, {
        new() {
            FittingDocumentId = Guid.NewGuid(),
            ClientId = PatchOp.Update(new Guid("3c1f4d5c-7ca9-4244-92ad-9db2795bf626"))
        },
        "ClientId.Value",
        "Client does not exist"
    }, {

```

```

        new() {
            FittingDocumentId = Guid.NewGuid(),
            FittingDate = PatchOp.Update(DateTimeOffset.MinValue)
        },
        "FittingDate.Value",
        "Planned fitting date should be greater than current day"
    }, {
        new() {
            FittingDocumentId = Guid.NewGuid(),
            EmployeeId = PatchOp.Update(1),
            DiscountCardCode = PatchOp.Update("0000000000000001"),
            ClientId = PatchOp.Update(new Guid("9b6619e0-3ef2-423e-bfc6-648c05cf4331")),
            FittingDate = PatchOp.Update(DateTimeOffset.UtcNow.AddDays(3))
        },
        null,
        "All fields set correctly"
    }, {
        new() {
            FittingDocumentId = Guid.NewGuid(),
            EmployeeId = PatchOp.Update(1),
            DiscountPercent = PatchOp.Update(20m),
            ClientId = PatchOp.Update(new Guid("9b6619e0-3ef2-423e-bfc6-648c05cf4331")),
            FittingDate = PatchOp.Update(DateTimeOffset.UtcNow.AddDays(3))
        },
        null,
        "All fields set correctly"
    }, {
        new() {
            FittingDocumentId = Guid.NewGuid(),
            DiscountCardCode = PatchOp.Remove("")
        },
        null,
        "Discount card code can be removed"
    }
}
};

```

```

[Theory]
[MemberData(nameof(ValidationData))]
public async Task Command_Should_BeValidatedCorrectly(
    UpdateFittingDocumentCommand command,
    string? property,
    string reason
) {
    // Act
    var sut = new UpdateFittingDocumentCommandValidator(_db.Object);
    var validationResult = await sut.TestValidateAsync(command);

    // Assert
    _testOutputHelper.WriteLine(reason);

    if (string.IsNullOrEmpty(property)) {
        validationResult.ShouldNotHaveAnyValidationErrors();
    } else {
        validationResult.ShouldHaveValidationErrorFor(property);
    }
}

```

Тестовый сценарий №3. Удаление документа примерки

Описание: Продавец хочет удалить существующий учет примерки в системе

Исходные данные: Тестовая база данных, существующий учет примерки

Шаги тестирования:

После успешной авторизации, продавец выбирает пункт «Примерки» в главном меню;
Продавец выбирает нужный документ, который необходимо удалить и нажимает на кнопку удаления.

После открывается окно подтверждения, где система спрашивать продавца, действительно ли он хочет удалить документ примерки: при подтверждении документ удаляется, при

отмене – нет.

Тестовые варианты:

Входные данные	Ожидаемый результат	Что проверяется
Идентификатор выбранного учета примерки из списка учетов примерок: 1	Документ примерки успешно удален!	Успешное удаление выбранного документа по его уникальному идентификатору.
<u>Примечание: подразумевается, что продавец не сможет задать идентификатор самостоятельно, или выбрать документ без идентификатора, но тем не менее, стоит рассмотреть случай, когда продавец пытается удалить документ примерки, которого нет в базе данных.</u> Идентификатор выбранной библиотеки из таблицы: 123	Документ не будет удален, поскольку его и так не существует в базе данных. Система продолжит работу в штатном режиме.	Ошибка удаления документа примерки по идентификатору, которого нет в базе данных. Проверка на устойчивость системы при возникновении внештатных ситуаций.

```
[Fact]
public async Task Should_ThrowException_When_DocumentIsNotNew() {
    // Arrange
    var documentId = new Guid("113039cc-bda2-4152-b5d1-aa700e4543ac");
    var command = new DeleteFittingDocumentCommand(documentId);

    _db.Setup(x => x.FittingDocuments)
        .ReturnsDbSet(
            [
                new() {
                    DocumentId = documentId,
                    StatusId = DocumentStatuses.PlannedFitting
                }
            ]
        );

    // Act
    var act = async () => await _sut.HandleAsync(command, new());

    // Assert
    await act.Should().ThrowAsync<CanNotChangeNotNewFittingDocumentException>();
}
```

```
[Fact]
public async Task Should_DeleteFittingDocumentWithItems() {
    // Arrange
    var documentId = new Guid("113039cc-bda2-4152-b5d1-aa700e4543ac");
    var command = new DeleteFittingDocumentCommand(documentId);

    _db.Setup(x => x.FittingDocuments)
        .ReturnsDbSet(
            [
                new() {
                    DocumentId = documentId,
                    StatusId = DocumentStatuses.New,
                    Products = [new(), new()]
                }
            ]
        );
```

```

    );

    // Act
    await _sut.HandleAsync(command, new());

    // Assert
    _db.Verify(x => x.Remove(It.IsAny<FittingDocument>()));
    _db.Verify(x => x.RemoveRange(It.IsAny<List<TradeDocumentProduct>>()));
    _db.Verify(x => x.SaveChangesAsync(It.IsAny<CancellationToken>()));
    _calendarEventService.Verify(x => x.RemoveTradeDocumentEventsAsync(documentId, It.IsAny<CancellationToken>()));
}

public static TheoryData<DeleteFittingDocumentCommand, string?, string> ValidationData => new() {
    {
        new(Guid.Empty),
        nameof(DeleteFittingDocumentCommand.FittingDocumentId),
        "Fitting document id is required"
    }, {
        new(new("0e319822-0c96-4c20-9b27-ac249a6f7959")),
        null,
        "All fields set correctly"
    }
};

[Theory]
[MemberData(nameof(ValidationData))]
public async Task Command_Should_BeValidatedCorrectly(
    DeleteFittingDocumentCommand command,
    string? property,
    string reason
) {
    // Arrange
    var sut = new DeleteFittingDocumentCommandValidator();

    // Act
    var validationResult = await sut.TestValidateAsync(command);

    // Assert
    _testOutputHelper.WriteLine(reason);

    if (string.IsNullOrEmpty(property)) {
        validationResult.ShouldNotHaveAnyValidationErrors();
    } else {
        validationResult.ShouldHaveValidationErrorFor(property);
    }
}

```

Тестовый сценарий №4. Сортировка списка документов примерки

Описание: Продавец хочет отсортировать отображающийся список документов примерки

Исходные данные: Тестовая база данных, существующий список учетов примерки, параметр для сортировки

Шаги тестирования:

После успешной авторизации, продавец выбирает пункт «Примерки» в главном меню; Продавец выбирает атрибут, по которому хочет провести сортировку списка, и нажимает на него. При первом нажатии будет произведена сортировка в возрастающем порядке, при втором нажатии будет произведена сортировка в убывающем, при третьем – сортировка по данному атрибуту будет убрана

Тестовые варианты:

Входные данные	Ожидаемый результат	Что проверяется
----------------	---------------------	-----------------

Серийный номер документа примерки	Документы будут отсортированы по их серийному номеру	Сортировка по числовым значениям.
Дата примерки	Документы будут отсортированы по дате примерки	Сортировка по значением типа дата.
ФИО Клиента	Документы будут отсортированы по клиентам	Сортировка по клиентам в алфавитном порядке.
Статус документа	Документы будут отсортированы по их статусам: новый, запланирован, завершен, отклонен	Сортировка по статусам.
Сумма	Документы будут отсортированы по общей сумме товаров в документах.	Сортировка по числовым значениям с плавающей запятой.

```
[Theory]
[InlineData("documents/fitting?page=1&pageSize=10")]
public async Task Should_GetFittingDocuments(
    string endpoint
) {
    // Act
    var response = await _httpClient.GetAsync(endpoint);

    // Assert
    response.Should().FailIfNotSuccessful();
}
```

Тестовый сценарий №5. Фильтрация списка документов примерки

Описание: Продавец хочет отсортировать отображающийся список документов примерки

Исходные данные: Тестовая база данных, существующий список учетов примерки

Шаги тестирования:

После успешной авторизации, продавец выбирает пункт «Примерки» в главном меню; Продавец выбирает атрибут, по которому хочет провести сортировку списка, и нажимает на него. При первом нажатии будет произведена сортировка в возрастающем порядке, при втором нажатии будет произведена сортировка в возрастающем, при третьем – сортировка по данному атрибуту будет убрана

Тестовые варианты:

Входные данные	Ожидаемый результат	Что проверяется
Серийный номер документа примерки	Документы будут отфильтрованы по введенному серийному номеру	Фильтрация по числовым значениям.
Дата примерки: от и до	Документы будут отфильтрованы по введенному диапазону дат	Фильтрация по значением типа дата.

ФИО Клиента	Документы будут отфильтрованы по клиентам	Фильтрация по клиентам в алфавитном порядке.
Статус документа	Документы будут отфильтрованы по их статусам: новый, запланирован, завершен, отклонен	Фильтрация по статусам.
Продавец	Документы будут отфильтрованы по ответственному продавцу.	Фильтрация по продавцу.

```
[Theory]
[InlineData("documents/fitting?page=1&pageSize=10")]
public async Task Should_GetFittingDocuments(
    string endpoint
) {
    // Act
    var response = await _httpClient.GetAsync(endpoint);

    // Assert
    response.Should().FailIfNotSuccessful();
}
```

3.3. Интеграционное тестирование

Совместимость среды разработки и операционной системы:

Протестирована совместимость .NET backend приложения и PostgreSQL базы данных на операционных системах Windows и Linux. Тестирование подтвердило, что система корректно работает на обеих платформах, без выявленных ошибок.

Интеграция с базой данных:

Проверена работа backend приложения с PostgreSQL. Все запросы к базе данных, включая запись, обновление, удаление и чтение, выполняются корректно. Данные отображаются и обрабатываются в соответствии с ожидаемыми результатами.

Взаимодействие компонентов системы:

Проведено тестирование взаимодействия между frontend (JavaScript), backend (C# API), и базой данных PostgreSQL. Все ключевые сценарии, включая:

- Создание, хранение, изменение и удаление записей учетов примерок;
- Отображение, фильтрация и сортировка списка учетов примерок.

Были успешно выполнены

3.4. Тестирование пользовательского интерфейса

Юзабилити-тестирование (Usability – удобство) – это проверка программного продукта на соответствие с требованиями в плане удобства использования приложения. Таким образом, с помощью юзабилити-тестирования мы можем определить эргономичность (приспособленность к использованию) программы.

Оценка удобства использования

по пятибалльной шкале, где:

- 5 — превосходно,
- 0 — требует значительных доработок

№	Вопрос	Комментарий пользователя	Оценка
1	Интуитивность интерфейса	Интерфейс понятный, легко ориентироваться.	5
2	Функциональные возможности	Все основные функции доступны, но не хватает подсказок для некоторых операций.	4
3	Скорость загрузки	В целом загрузка быстрая.	5
4	Возможность выбора разделов	Все разделы легко доступны, навигация интуитивно понятна.	5
5	Возможность быстрого ознакомления со структурой системы	Структура ясная, меню понятное, быстро осваивается.	4
6	Многоязычность	В наличии только русский язык, нужно добавить другие.	1
7	Оттенки основных цветов подобраны грамотно?	Цвета подобраны хорошо, нет раздражающих элементов.	4
8	Интуитивность интерфейса	Интерфейс понятный, но некоторые элементы не очевидны	3
9	Приятный шрифт	Шрифт удобочитаемый, не вызывает напряжения при длительном использовании.	5
10	Адаптивность интерфейса	Интерфейс не всегда корректно отображается на мобильных устройствах.	3
11	Наличие функции ночного режима	Функции ночного режима нет	0
12	Работает ли система с различными браузерами?	Работает стабильно в Google Chrome и Firefox.	5
13	Загрузка таблиц происходит достаточно быстро?	Таблицы загружаются довольно быстро.	4

14	Автоматическое обновление данных	Данные обновляются автоматически.	5
----	----------------------------------	-----------------------------------	---

3.5. Метрики кода

Результаты метрик кода						
Фильтр: Нет						
Иерархия	Индекс удобства поддержки	Сложность организации ц...	Глубина наследования	Взаимозависимость класс...	Строки исходного кода	Строки исполняемого кода
❏ Krista.Store.API.Tests (Release)	62	642	1	437	13 900	3 960
❏ AutoGeneratedProgram	100	1	1	1	1	0
❏ Krista.Store.API.Tests.WriteOffDocuments	62	15	1	65	285	83
❏ Krista.Store.API.Tests.TradeDocuments.ReservationDocuments	62	167	1	178	3 519	940
❏ Krista.Store.API.Tests.TradeDocuments.RentalDocuments	62	142	1	166	3 161	928
❏ Krista.Store.API.Tests.TradeDocuments.PurchaseReturnDocuments	61	26	1	90	502	170
❏ Krista.Store.API.Tests.TradeDocuments.InvoiceDocuments	62	38	1	103	782	253
❏ Krista.Store.API.Tests.TradeDocuments.FittingDocuments	62	118	1	158	2 486	665
❏ Krista.Store.API.Tests.MovementDocuments	59	18	1	82	445	131
❏ Krista.Store.API.Tests.Documents	64	6	1	45	99	25
❏ Krista.Store.API.Tests.Core	78	7	1	9	38	13
❏ Krista.Store.API.Tests.Clients	59	37	1	72	640	138
❏ Krista.Store.API.Tests.CalendarEvents	65	20	1	53	334	104
❏ Krista.Store.API.Tests.AuditDocuments	58	17	1	79	438	138
❏ Krista.Store.API.Tests.AcceptanceDocuments	62	15	1	65	282	82
❏ Krista.Store.API.Tests	50	15	1	40	878	290
❏ Krista.Store.API.E2e.Tests (Release)	67	254	2	297	5 278	1 670
❏ AutoGeneratedProgram	100	1	1	1	1	0
❏ Krista.Store.API.E2e.Tests.Endpoints.TradeDocuments	55	147	1	137	3 452	1 080
❏ Krista.Store.API.E2e.Tests.Endpoints	62	74	1	134	1 503	507
❏ Krista.Store.API.E2e.Tests.DbSetup	88	2	1	0	17	2
❏ Krista.Store.API.E2e.Tests.Core.Extensions	63	2	1	12	37	7
❏ Krista.Store.API.E2e.Tests.Core	78	26	2	99	237	67
❏ Krista.Store.API.E2e.Tests	83	2	1	19	31	7

Рисунок 2 Результаты метрик кода для тестов

Результаты метрик кода						
Фильтр: Нет						
Иерархия	Индекс удобства поддержки	Сложность организации ц...	Глубина наследования	Взаимозависимость класс...	Строки исходного кода	Строки исполняемого кода
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.UpdateFittingDocument	69	22	2	45	183	67
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.ScheduleFittingDocument	82	12	2	34	73	27
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.RemoveProductFromFitting	77	40	2	34	129	39
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.RemoveProductByBarcodeF	77	42	2	34	150	44
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.GetFittingDocuments	88	136	2	48	263	70
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.GetFittingDocumentArticul	93	7	1	19	33	5
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.GetFittingDocument	89	92	1	39	207	51
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.FinishFittingDocument	73	12	2	37	87	37
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.Exceptions	91	83	4	7	284	46
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.DeleteFittingDocument	81	8	2	35	66	23
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.CreateReservationForFitting	67	18	2	48	175	55
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.CreateInvoiceForFittingDoc	70	10	2	44	144	47
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.CreateFittingDocument	74	8	2	28	92	23
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.ChangeFittingProductPrice	72	20	2	30	80	33
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.CancelReservationForFitting	81	9	2	22	54	13
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.CancelFitting	72	15	2	36	97	28
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.AddProductToFittingDocu	79	36	2	39	140	39
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.AddProductByBarcodeToFit	78	52	2	42	195	46
❏ Krista.Store.API.Application.TradeDocuments.FittingDocuments.ActivateReservationForFittir	85	7	2	26	50	11

Рисунок 3 Результаты метрик кода для основного функционала управления учетами примерок

На основе предоставленных скриншотов таблиц метрик кода, можно сделать следующие выводы:

- 1. **Индекс удобства поддержки:**
 - Индекс удобства поддержки большинства методов и классов находится на достаточно высоком уровне (например, в районе 60–80). Это говорит о том, что код в целом читаем и поддерживаем.
- 2. **Сложность организации:**
 - Для многих методов сложность организации остаётся приемлемой (в пределах от 10 до 40). Однако есть методы с более высокой сложностью (например, GetFittingDocumentById и другие методы с показателем в районе 80–90). Это может указывать на необходимость рефакторинга.
- 3. **Глубина наследования:**
 - Для большинства классов глубина наследования остаётся на низком уровне (1–2), что положительно влияет на читаемость и упрощает понимание архитектуры.

4. Взаимозависимость классов:

- Взаимозависимость классов невелика (в основном 1–2), что указывает на хороший уровень декомпозиции и слабое зацепление между классами. Это упрощает тестирование и модификацию кода.

5. Строки исходного и исполняемого кода:

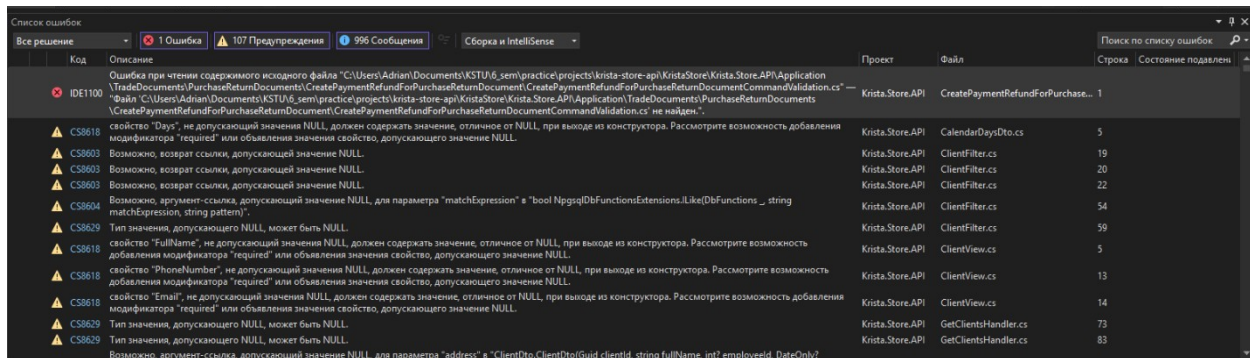
- В некоторых классах и методах количество строк исходного кода сравнительно велико, что может усложнить их сопровождение. Например, такие методы, как `UpdateFittingDocument` и `DeleteFittingDocument`, имеют заметное количество строк, что может потребовать их разделения на более мелкие части для упрощения сопровождения.

6. Общие рекомендации:

- Методы с высокой сложностью и большим количеством строк кода можно рассмотреть для рефакторинга с целью их декомпозиции.
- Сохранение низкого уровня зацепления и глубины наследования говорит о хорошей архитектурной структуре приложения.
- Учитывая, что индекс удобства поддержки довольно высокий, это позитивный показатель качества кода, но всегда есть возможность для улучшений, особенно в самых сложных и длинных методах.

3.7. Статический анализ кода

Статический анализ кода показал об 1 ошибке, 107 предупреждениях и 996 сообщениях.



Код	Описание	Проект	Файл	Строка	Состояние
IDe1100	Ошибка при чтении содержимого исходного файла "C:\Users\Adrian\Documents\KSTU\6_sem\practice\projects\krista-store-api\Kista.Store.API\Application\TradeDocuments\PurchaseReturnDocuments\CreatePaymentRefundForPurchaseReturnDocument\CreatePaymentRefundForPurchaseReturnDocumentCommandValidation.cs". Файл "C:\Users\Adrian\Documents\KSTU\6_sem\practice\projects\krista-store-api\Kista.Store.API\Application\TradeDocuments\PurchaseReturnDocuments\CreatePaymentRefundForPurchaseReturnDocument\CreatePaymentRefundForPurchaseReturnDocumentCommandValidation.cs" не найден.	Krista.Store.API	CreatePaymentRefundForPurchase...	1	
CS8618	свойство "Days", не допускающий значения NULL, должен содержать значение, отличное от NULL, при выходе из конструктора. Рассмотрите возможность добавления модификатора "required" или объявления значения свойство, допускающего значение NULL.	Krista.Store.API	CalendarDaysDto.cs	5	
CS9003	Возможно, возврат ссылки, допускающей значение NULL.	Krista.Store.API	ClientFilter.cs	19	
CS9003	Возможно, возврат ссылки, допускающей значение NULL.	Krista.Store.API	ClientFilter.cs	20	
CS9003	Возможно, возврат ссылки, допускающей значение NULL.	Krista.Store.API	ClientFilter.cs	22	
CS9004	Возможно, аргумент-ссылка, допускающий значение NULL, для параметра "matchExpression" в "bool NpgsqlDbFunctionsExtensions.Like(DbFunctions, string matchExpression, string pattern)".	Krista.Store.API	ClientFilter.cs	54	
CS8629	Тип значения, допускающего NULL, может быть NULL.	Krista.Store.API	ClientFilter.cs	59	
CS8618	свойство "FullName", не допускающий значения NULL, должен содержать значение, отличное от NULL, при выходе из конструктора. Рассмотрите возможность добавления модификатора "required" или объявления значения свойство, допускающего значение NULL.	Krista.Store.API	ClientView.cs	5	
CS8618	свойство "PhoneNumber", не допускающий значения NULL, должен содержать значение, отличное от NULL, при выходе из конструктора. Рассмотрите возможность добавления модификатора "required" или объявления значения свойство, допускающего значение NULL.	Krista.Store.API	ClientView.cs	13	
CS8618	свойство "Email", не допускающий значения NULL, должен содержать значение, отличное от NULL, при выходе из конструктора. Рассмотрите возможность добавления модификатора "required" или объявления значения свойство, допускающего значение NULL.	Krista.Store.API	ClientView.cs	14	
CS8629	Тип значения, допускающего NULL, может быть NULL.	Krista.Store.API	GetClientsHandler.cs	73	
CS8629	Тип значения, допускающего NULL, может быть NULL.	Krista.Store.API	GetClientsHandler.cs	83	

Рисунок 4 Первая часть ошибок и предупреждений

Список ошибок									
Все решения									
1 Ошибка107 Предупреждения996 СообщенияСборка в IntelliJ IDEA									
Поиск по списку ошибок									
СтрокаСостояние									
▲	C8618	Значение <code>articulName</code> , не допускающее значения <code>NULL</code> , должно содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	InvoiceDocumentArticulDoc.cs	104				
▲	C8619	свойство <code>"CurrencySymbol"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	ArticlePageInvoiceListDto.cs	104				
▲	C8660	Преобразование литерала, допускающего значение <code>NULL</code> или возможного значения <code>NULL</code> в <code>int</code> , не допускающий значение <code>NULL</code> .	Krinda.Store.API	GetInvoiceListByProductHandler.cs	115				
▲	C8660	Возможно, аргумент-ссылка, допускающий значение <code>NULL</code> , для параметра <code>"source"</code> в <code>"bool Enumerable.Contains<long>(IEnumerable<long> source, long value)"</code> .	Krinda.Store.API	GetInvoiceListByProductHandler.cs	131				
▲	C8662	Размещение вероятной пустой ссылки.	Krinda.Store.API	GetInvoiceListByProductHandler.cs	131				
▲	C8664	Возможно, аргумент-ссылка, допускающий значение <code>NULL</code> для параметра <code>"currentCurrencySymbol"</code> в <code>"ProductInvoiceDto.ProductInvoiceDto(int productid, int articulid, string articulName, int sizedIncl, string stringLineName, int colorId, string colorName, string colorHex, string currencySymbol, decimal price, int quantity, decimal totalSum, string productid)"</code> .	Krinda.Store.API	GetInvoiceListByProductHandler.cs	167				
▲	C8664	Возможно, аргумент-ссылка, допускающий значение <code>NULL</code> , для параметра <code>"serialNumbers"</code> в <code>"string ProductHelpers.GetProductUnit(int articulid, int sizedIncl, int colorId, List<long> serialNumbers)"</code> .	Krinda.Store.API	GetInvoiceListByProductHandler.cs	171				
▲	C8664	Возможно, аргумент-ссылка, допускающий значение <code>NULL</code> , для параметра <code>"pageIdList"</code> в <code>"PageInfo PageInfoExtensions.GetPage<InvoiceDocProductListGroupByArticulView>(PageIdList<InvoiceProductListGroupByArticulView> pageIdList)"</code> .	Krinda.Store.API	GetInvoiceListByProductHandler.cs	190				
▲	C8662	Размещение вероятной пустой ссылки.							
▲	C8662	Размещение вероятной пустой ссылки.							
▲	C8618	свойство <code>"ArticulName"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	ProductFilter.cs	24				
▲	C8618	свойство <code>"ArticulName"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	ProductFilter.cs	30				
▲	C8618	свойство <code>"Photo"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	ProductView.cs	7				
▲	C8618	свойство <code>"SizeInclName"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	ProductView.cs	9				
▲	C8618	свойство <code>"ColorName"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	ProductView.cs	11				
▲	C8618	свойство <code>"ColorHex"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	ProductView.cs	12				
▲	C8618	свойство <code>"ArticulName"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	ProductView.cs	21				
▲	C8618	свойство <code>"Products"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	ProductView.cs	24				
▲	C8618	свойство <code>"ArticulName"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	RentalDocumentArticulDoc.cs	5				
▲	C8618	свойство <code>"ChildDocuments"</code> , не допускающий значения <code>NULL</code> , должен содержать значение, отличное от <code>NULL</code> , при выходе из конструктора. Рассмотрите возможность добавления модификатора <code>"required"</code> или объявления значения свойства, допускающего значение <code>NULL</code> .	Krinda.Store.API	RentalDocumentArticulDoc.cs	10				

Рисунок 5 Вторая часть ошибок и предупреждений

Список ошибок

Все решения

1 Ошибка

107 Предупреждения

996 Сообщения

Сборка и IntelliSense

Поиск по списку ошибок

Строка

Состояние подавлен

Код	Описание	Проект	Файл	Строка	Состояние подавлен
IDE0290	Использовать основной конструктор	Krista.Store.API	GetCountOfCalendarEventsTodayH...	12	
IDE0290	Использовать основной конструктор	Krista.Store.API	CalendarEventService.cs	9	
IDE0290	Использовать основной конструктор	Krista.Store.API	UpdateCustomCalendarEventHandl...	20	
IDE0290	Использовать основной конструктор	Krista.Store.API	CreateClientHandler.cs	12	
CA1822	Член "GetContactInfos" не обращается к данным экземпляров, поэтому его можно помечать как статический.	Krista.Store.API	CreateClientHandler.cs	62	
IDE0290	Использовать основной конструктор	Krista.Store.API	CreatedClientDto.cs	7	
CA1860	Для ясности и для обеспечения производительности старайтесь сравнивать 'Length' с 0 вместо того, чтобы использовать 'Any()'	Krista.Store.API	ClientFilter.cs	57	
IDE0290	Использовать основной конструктор	Krista.Store.API	ClientsDto.cs	7	
IDE0290	Использовать основной конструктор	Krista.Store.API	ClientsDto.cs	33	
IDE0290	Использовать основной конструктор	Krista.Store.API	GetClientHandler.cs	14	
IDE0290	Использовать основной конструктор	Krista.Store.API	ClientLookupDto.cs	7	
IDE0290	Использовать основной конструктор	Krista.Store.API	GetClientLookupHandler.cs	10	
IDE0290	Использовать основной конструктор	Krista.Store.API	GetClientLookupQuery.cs	6	
CA1860	Для ясности и для обеспечения производительности старайтесь сравнивать 'Length' с 0 вместо того, чтобы использовать 'Any()'	Krista.Store.API	DiscountCardFilter.cs	66	
IDE0290	Использовать основной конструктор	Krista.Store.API	DiscountCardsDto.cs	8	
IDE0290	Использовать основной конструктор	Krista.Store.API	GetDiscountCardsHandler.cs	13	
IDE0060	Удалить неиспользуемый параметр 'clientId', если он не является частью предоставляемого общедоступного API	Krista.Store.API	UpdatedClientDto.cs	6	
CA2211	Поля, не являющиеся константами, не должны быть видимыми	Krista.Store.API	FileUriSources.cs	4	
CA2211	Поля, не являющиеся константами, не должны быть видимыми	Krista.Store.API	FileUriSources.cs	5	
IDE0290	Использовать основной конструктор	Krista.Store.API	DirectoryPathShouldNotContainFil...	8	
IDE0290	Использовать основной конструктор	Krista.Store.API	FileAlreadyExistException.cs	9	
IDE0300	Инициализацию коллекции можно упростить.	Krista.Store.API	FileAlreadyExistException.cs	16	
IDE0290	Использовать основной конструктор	Krista.Store.API	FilePathInfoForbiddenDirectoryExce...	8	
IDE0290	Использовать основной конструктор	Krista.Store.API	FilePathShouldContainFileNameExc...	8	
IDE0290	Использовать основной конструктор	Krista.Store.API	UploadFileNotFoundException.cs	9	
IDE0300	Инициализацию коллекции можно упростить.	Krista.Store.API	UploadFileNotFoundException.cs	16	
IDE0290	Использовать основной конструктор	Krista.Store.API	UploadFileService.cs	14	
IDE0208	Инициализацию коллекции можно упростить.	Krista.Store.API	UploadFileService.cs	18	

Рисунок 6 Третья часть ошибок и предупреждений

По итогам статического анализа:

- 1 ошибка связана с ошибкой при чтении содержимого исходного файла;
- 107 предупреждений, подавляющее большинство из которых были связаны со спецификой работы с Nullable типом данных;
- 996 сообщений, в основном советующие использование альтернативного способа написания тех или иных функций или общие советы к коду.

На основе предоставленных скриншотов сделаны следующие выводы:

1. Кодовые метрики и сложность кода:

- Проект имеет умеренные показатели "Индекса удобства поддержки" и "Сложности организации". Большинство классов и методов имеют сравнительно низкие уровни глубины наследования и классовой взаимозависимости, что говорит о приемлемой модульности.

- Отмечаются области с высокой сложностью методов, особенно связанные с обработкой документов `fitting` и `trade`, что может свидетельствовать о возможной необходимости рефакторинга для улучшения читаемости и поддержки.

2. Использование ресурсов:

- Процесс API сервиса демонстрирует существенное использование памяти и процессорного времени. Наибольшее время процессора занято методами Entity Framework (`ToListAsync`, `DbContext`), что может указывать на необходимость оптимизации запросов к базе данных.
- Основная нагрузка на процессор связана с выполнением .NET Core библиотек и внутренними функциями базы данных, что ожидаемо для высоконагруженного приложения.

3. Список ошибок и предупреждений:

- **Ошибки (Error):**
 - Выявлена критическая ошибка, связанная с отсутствием исходного файла в проекте. Это может быть результатом неверной настройки проекта или отсутствия файла в репозитории.
- **Предупреждения (Warning):**
 - Значительное количество предупреждений связано с использованием свойств или параметров, допускающих `null`. Рекомендуется рассмотреть внедрение строгой проверки на nullability или использование `nullable reference types`.
 - Появляются предупреждения о возможной оптимизации кода, например, переиспользование конструкторов или упрощение методов.
- **Информационные сообщения (Info):**
 - Сообщения подчеркивают места, где возможно упростить код, заменить неэффективные операции, и предлагают использовать более подходящие конструкции языка.

4. Рекомендации:

- **Оптимизация производительности:**
 - Пересмотреть запросы Entity Framework, чтобы уменьшить избыточные вызовы и нагрузку на CPU.
 - Рассмотреть кэширование часто запрашиваемых данных.
- **Качество кода:**
 - Использовать статические анализаторы кода и автоматизацию проверки на `null` для сокращения количества предупреждений.
 - Выполнить рефакторинг сложных методов и классов, чтобы улучшить их читаемость и поддержку.
- **Управление ошибками:**
 - Обеспечить наличие всех исходных файлов проекта в репозитории, проверить настройки CI/CD.
 - Устранить критические ошибки сборки для стабильности приложения.

Этот анализ помогает определить ключевые точки улучшения и повысить общую эффективность и качество разработки проекта.

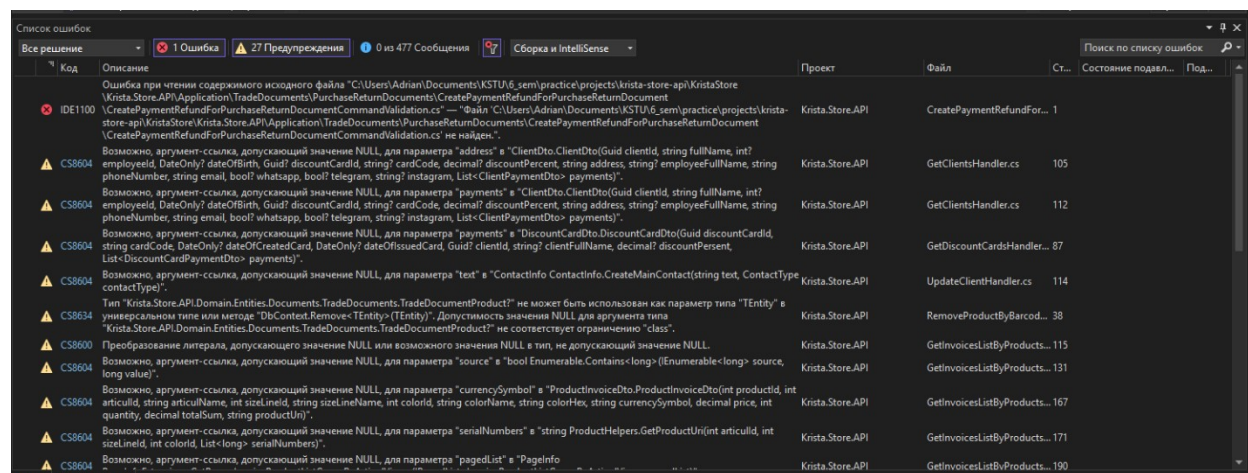


Figure 1 Результат подавления предупреждений

После процесса подавления ошибок и предупреждений, их количество было очень сильно снижено. Количество предупреждений было уменьшено со 107 до 27, количество сообщений было уменьшено с 997 до 477.

3.8. Профилирование программы

Профилирование программы проведено с помощью инструментов Microsoft Visual Studio Enterprise Edition 2022.

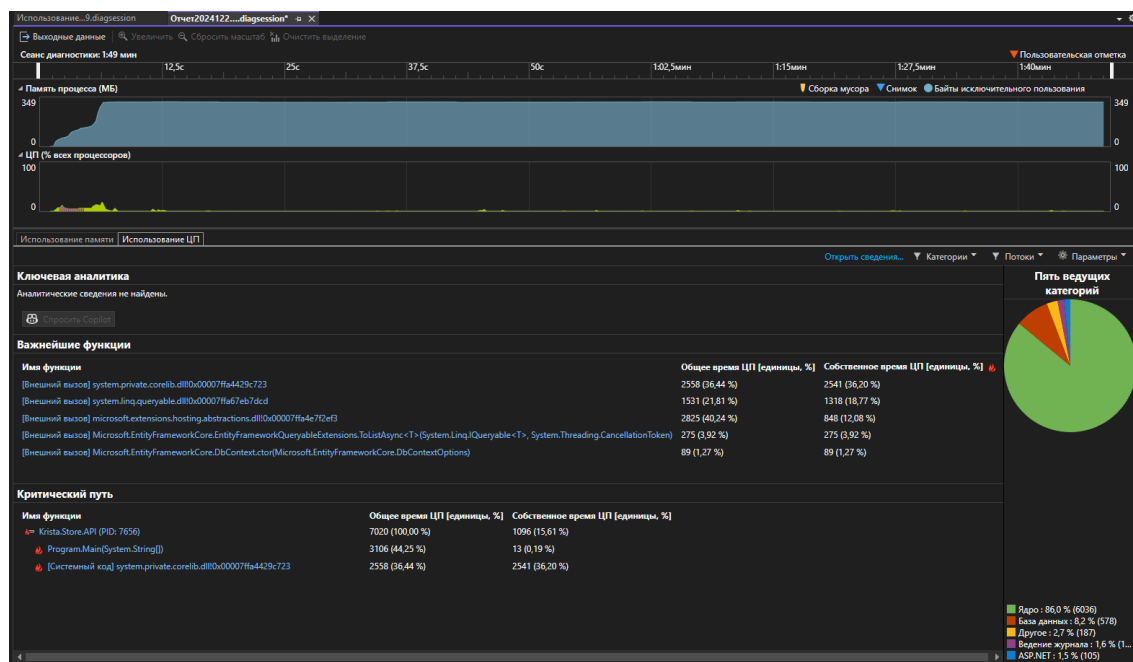


Рисунок 7 Отчет по тестированию производительности

Общее время ЦП [единицы, %]	Собственное время ЦП [единицы, %] 
2558 (36,44 %)	2541 (36,20 %)
1531 (21,81 %)	1318 (18,77 %)
2825 (40,24 %)	848 (12,08 %)
275 (3,92 %)	275 (3,92 %)
89 (1,27 %)	89 (1,27 %)

Рисунок 8 Подробный ответ по тестированию производительности с упором на время ЦП

Критический путь		
Имя функции	Общее время ЦП [единицы, %]	Собственное время ЦП [единицы, %]
 Krista.Store.API (PID: 7656)	7020 (100,00 %)	1096 (15,61 %)
 Program.Main(System.String[])	3106 (44,25 %)	13 (0,19 %)
 [Системный код] system.private.corelib.dll!0x00007ffa4429c723	2558 (36,44 %)	2541 (36,20 %)

Рисунок 9 Найденные критические пути

На основе предоставленного скриншота, можно сделать следующие выводы:

1. Использование памяти:

- Память процесса постепенно увеличивается и достигает примерно **349 МБ**. Это может указывать на накопление данных в процессе работы, возможные утечки памяти или недостаточную оптимизацию кода.

2. ЦПУ:

- Нагрузка на ЦП относительно невысока, остаётся в пределах **2–12%**. Это может свидетельствовать о том, что процесс не является CPU-интенсивным, однако важно проверить, что это не вызвано избыточными задержками в других частях системы (например, ожиданием ввода-вывода).

3. Важнейшие функции:

- Наибольшее время затрачивается на следующие функции:
 - `System.Private.CoreLib.dll` (внешний вызов): **2538 единиц (36.4%)**. Это указывает на то, что значительное время уходит на внутренние системные операции.
 - `Microsoft.EntityFrameworkCore.Query.QueryableExtensions.ToListAsync<T>`: **2825 единиц (40.4%)**, включая задержки в базе данных. Это говорит о потенциально медленных запросах к базе данных, связанных с использованием Entity Framework.
 - `Microsoft.EntityFrameworkCore.DbContext.Dispose`: **275 единиц (3.92%)**. Это может быть связано с частым созданием и уничтожением контекстов базы данных.

4. Критический путь:

- Основное время выполнения (70%) связано с методом `Krista.Store.API`, который включает вызовы запросов через Entity Framework. Вероятно, здесь кроется узкое место в производительности.

- `Program.Main(String[])` затрачивает **18.7%** времени, что указывает на возможные проблемы в стартовой логике приложения.

Часть 4. Заключение

В результате проведенного тестирования системы автоматизации управления розничным магазином одежды были достигнуты существенные результаты. Комплексный подход, включающий 23 тестовых случаев, из которых успешно пройдено 21, позволил оценить надежность и функциональные возможности системы.

В процессе тестирования выявлено два дефекта: ошибка при фильтрации и сортировки документов примерок по их датам примерки. Оба дефекта были классифицированы как незначительные, соответственно, не влияют критически на работоспособность системы, но требует доработки для повышения удобства и функциональности.

Кроме того, было предложено несколько улучшений: доработка валидации входных данных, оптимизация горячей путей кода, подавление оставшихся сообщений и предупреждений после статического анализа. Эти изменения могут существенно повысить пользовательский опыт и удобство работы с системой.

Таким образом, проведенное тестирование подтверждает стабильность и общее качество разработки системы, при этом выявленные области для оптимизации и доработок задают направления для ее дальнейшего совершенствования.